메트릭 수집 및 알림 시스템 분석

개요

Omni Monitor 서비스는 Prometheus 기반의 포괄적인 메트릭 수집 시스템을 구현하여 네트워크의 모든 측면을 실시간으로 모니터링합니다. 각 컴포넌트별로 특화된 메트릭을 수집하고, 임계값 기반 알림 시스템을 통해 문제 상황을 즉시 감지합니다.

메트릭 아키텍처

전체 메트릭 구조

```
Prometheus Metrics System
├── monitor_* # Monitor 서비스 자체 메트릭
| ├── rpc_* # RPC 동기화 메트릭
| ├── cprovider_* # 합의 Provider 메트릭
| └── public_rpc_* # 퍼블릭 RPC 메트릭
├── monitor_xchain_* # 크로스체인 메트릭
| ├── emit_stream_offset # 메시지 발행 오프셋
| ├── submit_stream_offset # 메시지 제출 오프셋
| ├── head_height # 체인 헤드 높이
| └── halo_* # Halo 증명 메트릭
├── monitor_validator_* # 검증자 메트릭
| ├── power # 검증자 파워
```

```
│ ├── jailed # 감옥 상태
│ ├── bonded # 본딩 상태
│ └── uptime # 업타임
├── monitor_account_* # 계정 메트릭
│ ├── balance_ether # 네이티브 토큰 잔액
│ ├── token_balance_ether # ERC20 토큰 잔액
│ └── balance_low # 잔액 부족 알림
├── monitor_contract_* # 컨트랙트 메트릭
│ ├── balance_ether # 컨트랙트 잔액
│ ├── balance_token # 토큰 잔액
│ └── balance_low/high # 임계값 알림
├── monitor_staking_* # 스테이킹 메트릭
│ ├── avg_rewards_percentage # 평균 보상률
│ ├── delegators # 위임자 수
│ └── stake_* # 스테이크 통계
└── monitor_supply_* # 토큰 공급량 메트릭
├── cchain_total_ether # 합의체인 공급량
├── l1_erc20_total_ether # L1 ERC20 공급량
└── l1_bridge_balance_ether # 브릿지 잔액
```

컴포넌트별 메트릭 분석

1. 크로스체인 메트릭 (xmonitor/metrics.go)

크로스체인 메시지 처리와 관련된 핵심 메트릭들입니다.

메시지 오프셋 메트릭

```
var (
// 메시지 발행 오프셋 - 소스 체인에서 발행된 메시지 오프셋
emitMsgOffset = promauto.NewGaugeVec(prometheus.GaugeOpts{
Namespace: "monitor",
Subsystem: "xchain",
Name: "emit_stream_offset",
Help: "The latest emitted xmsg offset on a source chain for a specific destinati
}, []string{"stream"})
// 메시지 제출 오프셋 - 목적지 체인에 제출된 메시지 오프셋
submitMsgOffset = promauto.NewGaugeVec(prometheus.GaugeOpts{
Namespace: "monitor",
Subsystem: "xchain",
Name: "submit_stream_offset",
Help: "The latest submitted xmsg stream offset on a destination chain for a spec
}, []string{"stream"})
// 증명 제출 오프셋 - 목적지 체인에 제출된 증명 오프셋
submitAttestOffset = promauto.NewGaugeVec(prometheus.GaugeOpts{
Namespace: "monitor",
Subsystem: "xchain",
Name: "submit_attest_offset",
```

```
Help: "The latest submitted attest offset on a destination chain for a specific
}, []string{"stream"})
)
```

블록 헤드 메트릭

```
// 체인별 헤드 높이 추적
headHeight = promauto.NewGaugeVec(prometheus.GaugeOpts{
Namespace: "monitor",
Subsystem: "xchain",
Name: "head_height",
Help: "The latest height of different types of head blocks on a specific chain",
}, []string{"chain", "type"}) // type: latest, safe, finalized
```

Halo 증명 메트릭

```
// Halo에서 증명된 높이
attestedHeight = promauto.NewGaugeVec(prometheus.GaugeOpts{
Namespace: "monitor",
Subsystem: "xchain",
Name: "halo_attested_height",
Help: "The latest halo attested height of a specific chain version",
}, []string{"chain_version"})
```

```go
// Halo 증명 오프셋
attestedOffset = promauto.NewGaugeVec(prometheus.GaugeOpts{
Namespace: "monitor",
Subsystem: "xchain",
Name: "halo_attest_offset",
Help: "The latest halo attest offset of a specific chain version",
}, []string{"chain_version"})
```

2. 검증자 메트릭 (validator/metrics.go)

검증자 상태와 성능을 추적하는 메트릭들입니다.

검증자 상태 메트릭

```go
var (
// 검증자 파워 (잠재적 합의 파워)
powerGauge = promutil.NewResetGaugeVec(prometheus.GaugeOpts{
Namespace: "monitor",
Subsystem: "validator",
Name: "power",
Help: "Current potential power by validator (may not be bonded)",
}, []string{"validator", "validator_address", "operator"})
// 감옥 상태 (1=감옥됨, 0=정상)
jailedGauge = promutil.NewResetGaugeVec(prometheus.GaugeOpts{
```

```
Namespace: "monitor",
Subsystem: "validator",
Name: "jailed",
Help: "Constant gauge set to 1 if the validator is jailed otherwise 0",
}, []string{"validator"})
// 본딩 상태 (1=본딩됨, 0=언본딩됨)
bondedGauge = promutil.NewResetGaugeVec(prometheus.GaugeOpts{
Namespace: "monitor",
Subsystem: "validator",
Name: "bonded",
Help: "Constant gauge set to 1 if the validator is bonded otherwise 0",
}, []string{"validator"})
)
```

검증자 성능 메트릭

```
// 업타임 (슬래싱 윈도우 내 서명 비율)
uptimeGauge = promutil.NewResetGaugeVec(prometheus.GaugeOpts{
Namespace: "monitor",
Subsystem: "validator",
Name: "uptime",
Help: "Percentage of blocks signed in past <slashing_signing_window>",
}, []string{"validator"})
```

```go
// 검증자 보상
rewardsGauge = promutil.NewResetGaugeVec(prometheus.GaugeOpts{
Namespace: "monitor",
Subsystem: "validator",
Name: "rewards",
Help: "Validator rewards",
}, []string{"validator"})
```

검증자 데이터 수집 과정

```go
type sample struct {
ConsensusEthAddr common.Address // 합의 이더리움 주소
ConsensusCmtAddr crypto.Address // 합의 CometBFT 주소
OperatorEthAddr common.Address // 운영자 이더리움 주소
Power  int64  // 검증자 파워
Jailed  bool  // 감옥 상태
Bonded  bool  // 본딩 상태
Tombstoned  bool  // 툼스톤 상태
Uptime  float64  // 업타임 비율
Rewards  float64  // 보상 금액
}

func  sampleValidator(s sample) {
```

```
powerGauge.WithLabelValues(s.ConsensusEthAddr.String(), s.ConsensusCmtAddr.Strin
jailedGauge.WithLabelValues(s.ConsensusEthAddr.String()).Set(boolToFloat(s.Jaile
bondedGauge.WithLabelValues(s.ConsensusEthAddr.String()).Set(boolToFloat(s.Bonde
uptimeGauge.WithLabelValues(s.ConsensusEthAddr.String()).Set(s.Uptime)
rewardsGauge.WithLabelValues(s.ConsensusEthAddr.String()).Set(s.Rewards)
}
```

3. 계정 메트릭 (account/metrics.go)

네트워크 참여자들의 계정 상태를 모니터링합니다.

계정 잔액 메트릭

```
var (
// 네이티브 토큰 잔액 (이더 단위)
accountBalance = promauto.NewGaugeVec(prometheus.GaugeOpts{
Namespace: "monitor",
Subsystem: "account",
Name: "balance_ether",
Help: "The native balance of the account on a specific chain in ether. Alert if
}, []string{"chain", "role"})
// 계정 nonce
accountNonce = promauto.NewGaugeVec(prometheus.GaugeOpts{
Namespace: "monitor",
```

```
Subsystem: "account",

Name: "nonce",

Help: "The nonce of the account on a specific chain",

}, []string{"chain", "role"})

// 잔액 부족 알림 (1=부족, 0=충분)

accountBalanceLow = promauto.NewGaugeVec(prometheus.GaugeOpts{

Namespace: "monitor",

Subsystem: "account",

Name: "balance_low",

Help: "Constant gauge indicating whether the account balance is below the minimu

}, []string{"chain", "role"})

)
```

ERC20 토큰 메트릭

```
// ERC20 토큰 잔액

tokenBalance = promauto.NewGaugeVec(prometheus.GaugeOpts{

Namespace: "monitor",

Subsystem: "account",

Name: "token_balance_ether",

Help: "The token balance of the account on a specific chain in ether",

}, []string{"chain", "role", "token"})
```

```
// 토큰 잔액 부족 알림
tokenBalanceLow = promauto.NewGaugeVec(prometheus.GaugeOpts{
Namespace: "monitor",
Subsystem: "account",
Name: "token_balance_low",
Help: "Constant gauge indicating whether the token balance is below the minimum
}, []string{"chain", "role", "token"})
```

4. 스마트 컨트랙트 메트릭 (contract/metrics.go)

스마트 컨트랙트의 상태와 잔액을 모니터링합니다.

컨트랙트 잔액 메트릭

```
var (
// 컨트랙트 네이티브 토큰 잔액
contractBalance = promauto.NewGaugeVec(prometheus.GaugeOpts{
Namespace: "monitor",
Subsystem: "contract",
Name: "balance_ether",
Help: "The balance of the contract on a specific chain in ether. Alert if low or
}, []string{"chain", "name"})
// 컨트랙트 ERC20 토큰 잔액
contractTokenBalance = promauto.NewGaugeVec(prometheus.GaugeOpts{
Namespace: "monitor",
```

```
Subsystem: "contract",

Name: "balance_token",

Help: "The balance of the contract of specific ERC20 tokens on a specific chain.

}, []string{"chain", "name", "token_symbol", "token_address"})

)
```

임계값 알림 메트릭

```
// 잔액 부족 알림

contractBalanceLow = promauto.NewGaugeVec(prometheus.GaugeOpts{

Namespace: "monitor",

Subsystem: "contract",

Name: "balance_low",

Help: "Constant gauge indicating whether the contract balance is below the minim

}, []string{"chain", "name"})


// 잔액 과다 알림

contractBalanceHigh = promauto.NewGaugeVec(prometheus.GaugeOpts{

Namespace: "monitor",

Subsystem: "contract",

Name: "balance_high",

Help: "Constant gauge indicating whether the contract balance is above the maxim
```

```
}, []string{"chain", "name"})
```

5. 스테이킹 메트릭 (staking/metrics.go)

네트워크 스테이킹 상태와 보상을 추적합니다.

스테이킹 통계 메트릭

```
var (
// 평균 스테이킹 보상률 (0-1 범위)
rewardsAvg = promauto.NewGauge(prometheus.GaugeOpts{
Namespace: "monitor",
Subsystem: "staking",
Name: "avg_rewards_percentage",
Help: "Average staking rewards percentage [0,1]",
})
// 위임자 수
delegatorsCount = promauto.NewGauge(prometheus.GaugeOpts{
Namespace: "monitor",
Subsystem: "staking",
Name: "delegators",
Help: "Number of unique delegators",
})
// 평균 스테이크 크기 (이더 단위)
stakeAvg = promauto.NewGauge(prometheus.GaugeOpts{
```

```
Namespace: "monitor",
Subsystem: "staking",
Name: "stake_avg_ether",
Help: "Average stake size in ether",
})
// 중간값 스테이크 크기 (이더 단위)
stakeMedian = promauto.NewGauge(prometheus.GaugeOpts{
Namespace: "monitor",
Subsystem: "staking",
Name: "stake_median_ether",
Help: "Median stake size in ether",
})
)
```

6. 토큰 공급량 메트릭 (supply/metrics.go)

OMNI 토큰의 공급량과 분배를 추적합니다.

공급량 메트릭

```
var (
// 합의체인 토큰 공급량
cChainSupply = promauto.NewGauge(prometheus.GaugeOpts{
Namespace: "monitor",
```

```
Subsystem: "supply",
Name: "cchain_total_ether",
Help: "Token supply on the consensus chain in ether",
})
// L1 ERC20 토큰 공급량
l1Erc20Supply = promauto.NewGauge(prometheus.GaugeOpts{
Namespace: "monitor",
Subsystem: "supply",
Name: "l1_erc20_total_ether",
Help: "OMNI ERC20 token supply on Ethereum chain in ether",
})
// L1 브릿지 잔액
bridgeBalance = promauto.NewGauge(prometheus.GaugeOpts{
Namespace: "monitor",
Subsystem: "supply",
Name: "l1_bridge_balance_ether",
Help: "OMNI ERC20 token balance of the bridge contract on Ethereum in ether",
})
)
```

7. Monitor 서비스 자체 메트릭 (app/metrics.go)

Monitor 서비스 자체의 성능과 상태를 추적합니다.

동기화 메트릭

```go
var (
// RPC 동기화 차이
syncDiff = promauto.NewGaugeVec(prometheus.GaugeOpts{
Namespace: "monitor",
Subsystem: "rpc",
Name: "sync_diff",
Help: "Maximum sync difference (concurrent latest heights) per chain",
}, []string{"chain"})
// 퍼블릭 RPC 동기화 차이
publicRPCSyncDiff = promauto.NewGauge(prometheus.GaugeOpts{
Namespace: "monitor",
Subsystem: "public_rpc",
Name: "sync_diff",
Help: "Sync difference (highest blocks) betweent public RPC and omni node",
})
)
```

성능 메트릭

```go
// 히스토리컬 베이스라인 성능
histBaseline = promauto.NewHistogramVec(prometheus.HistogramOpts{
Namespace: "monitor",
Subsystem: "cprovider",
```

```
Name: "historical_baseline_seconds",

Help: "Baseline time (in seconds) to stream historical approved attestation",

Buckets: prometheus.ExponentialBucketsRange(time.Second.Seconds(), time.Hour.Sec

}, []string{"chain", "size"})


// 가스 팁 캡 (Gwei 단위)

gasTipCap = promauto.NewGauge(prometheus.GaugeOpts{

Namespace: "monitor",

Subsystem: "omni_evm",

Name: "gas_tip_cap_gwei",

Help: "Suggested OmniEVM gas tip cap in gwei",

})
```

업그레이드 추적 메트릭

```
// 계획된 업그레이드

plannedUpgradeGauge = promutil.NewResetGaugeVec(prometheus.GaugeOpts{

Namespace: "monitor",

Name: "planned_upgrade",

Help: "Height of current planned (non-processed) upgrade by name",

}, []string{"upgrade"})


// 적용된 업그레이드
```

```
appliedUpgradeGauge = promutil.NewResetGaugeVec(prometheus.GaugeOpts{
Namespace: "monitor",
Name: "applied_upgrade",
Help: "Height of last applied (processed) upgrade by name",
}, []string{"upgrade"})
```

메트릭 수집 패턴

1. 주기적 수집 패턴

대부분의 메트릭은 주기적으로 수집됩니다:

```
func monitorForever(ctx context.Context, ...) {
ticker := time.NewTicker(time.Second * 30) // 30초 간격
defer ticker.Stop()
for {
select {
case <-ctx.Done():
return
case <-ticker.C:
err := monitorOnce(ctx, ...)
if err != nil {
log.Error(ctx, "Monitoring failed (will retry)", err)
}
```

```
}
}
}
```

2. 이벤트 기반 수집 패턴

일부 메트릭은 이벤트 발생 시 즉시 업데이트됩니다:

```
func monitorAttestedForever(ctx context.Context, chainVer xchain.ChainVersion,
var lastAttestOffset uint64
for {
att, ok, err := cprovider.LatestAttestation(ctx, chainVer)
if att.AttestOffset == lastAttestOffset {
continue // 변화가 없으면 스킵
}
// 새로운 증명이 있을 때만 메트릭 업데이트
attestedHeight.WithLabelValues(chainVerName).Set(float64(att.BlockHeight))
attestedOffset.WithLabelValues(chainVerName).Set(float64(att.AttestOffset))
lastAttestOffset = att.AttestOffset
}
}
```

알림 시스템

1. 임계값 기반 알림

각 메트릭은 임계값을 기반으로 알림을 생성합니다:

잔액 부족 알림

```
func monitorAccountOnce(ctx context.Context, account eoa.Account, ...) error {
balance, err := client.BalanceAt(ctx, account.Address, nil)
if err != nil {
return err
}
// 잔액 메트릭 설정
accountBalance.WithLabelValues(chainName, string(account.Role)).Set(bi.ToEtherF6
// 임계값 확인 및 알림 메트릭 설정
thresholds, ok := eoa.GetFundThresholds(meta.NativeToken, network, account.Role)
if ok {
var isLow float64
if bi.LT(balance, thresholds.MinBalance()) {
isLow = 1 // 알림 상태
}
accountBalanceLow.WithLabelValues(chainName, string(account.Role)).Set(isLow)
}
return nil
}
```

컨트랙트 잔액 알림

```
func monitorContractOnce(ctx context.Context, contract contracts.Contract, ...)
balance, err := client.BalanceAt(ctx, contract.Address, nil)
if err != nil {
return err
}
contractBalance.WithLabelValues(chain.Name, contract.Name).Set(bi.ToEtherF64(bal
// 최소 임계값 확인
if contract.FundThresholds != nil {
var isLow float64
if bi.LTE(balance, contract.FundThresholds.MinBalance()) {
isLow = 1
}
contractBalanceLow.WithLabelValues(chain.Name, contract.Name).Set(isLow)
}
// 최대 임계값 확인
if contract.WithdrawThresholds != nil {
var isHigh float64
if bi.GTE(balance, contract.WithdrawThresholds.MaxBalance()) {
isHigh = 1
}
contractBalanceHigh.WithLabelValues(chain.Name, contract.Name).Set(isHigh)
```

```
}
return nil
}
```

2. 상태 변화 알림

검증자 상태 변화 등을 추적합니다:

```
func sampleValidator(s sample) {
// 불린 값을 float64로 변환 (1=true, 0=false)
jailedGauge.WithLabelValues(s.ConsensusEthAddr.String()).Set(boolToFloat(s.Jaile
bondedGauge.WithLabelValues(s.ConsensusEthAddr.String()).Set(boolToFloat(s.Bonde
tombstonedGauge.WithLabelValues(s.ConsensusEthAddr.String()).Set(boolToFloat(s.T
}

func boolToFloat(b bool) float64 {
if !b {
return 0
}
return 1
}
```

메트릭 유틸리티

ResetGaugeVec

동적 라벨을 가진 메트릭의 메모리 누수를 방지하는 유틸리티입니다:

```
type ResetGaugeVec struct {
inner *prometheus.GaugeVec
mu sync.Mutex
labels  map[string]bool  // 설정된 라벨 추적
}

func (g *ResetGaugeVec) WithLabelValues(lvs ...string) prometheus.Gauge {
g.mu.Lock()
defer  g.mu.Unlock()
// 라벨 조합을 추적
g.labels[strings.Join(lvs, separator)] = true
return  g.inner.WithLabelValues(lvs...)
}

func (g *ResetGaugeVec) Reset(lvs ...string) {
g.mu.Lock()
defer  g.mu.Unlock()
// 매칭되는 라벨들을 삭제
for  label := range  g.labels {
if  matchesFilter(label, lvs) {
```

```
g.inner.DeleteLabelValues(strings.Split(label, separator)...)
delete(g.labels, label)
}
}
}
```

사용 예시:

```
// 검증자가 제거될 때 관련 메트릭 정리
powerGauge.Reset()
jailedGauge.Reset()
bondedGauge.Reset()
```

메트릭 엔드포인트

HTTP 서버 설정

Monitor 서비스는 다음 엔드포인트를 제공합니다:

```
func serveMonitoring(address string) <-chan error {
mux := http.NewServeMux()
// Prometheus 메트릭 엔드포인트
mux.Handle("/metrics", promhttp.Handler())
// pprof 디버깅 엔드포인트
mux.HandleFunc("/debug/pprof/", pprof.Index)
```

```go
mux.HandleFunc("/debug/pprof/profile", pprof.Profile)
mux.HandleFunc("/debug/pprof/trace", pprof.Trace)
srv := &http.Server{
Addr: address, // 기본값: :26660
ReadHeaderTimeout: 5 * time.Second,
IdleTimeout: 5 * time.Second,
WriteTimeout: 5 * time.Second,
Handler: mux,
}
return srv.ListenAndServe()
}
```

메트릭 접근

- **Prometheus 메트릭**: `http://monitor:26660/metrics`
- **성능 프로파일**: `http://monitor:26660/debug/pprof/profile`
- **메모리 프로파일**: `http://monitor:26660/debug/pprof/heap`

에러 처리 및 복구

메트릭 수집 실패 처리

```go
func monitorForever(ctx context.Context, ...) {
ticker := time.NewTicker(time.Second * 30)
defer ticker.Stop()
```

```go
for {
select {
case <-ctx.Done():
return
case <-ticker.C:
err := monitorOnce(ctx, ...)
if ctx.Err() != nil {
return // 컨텍스트 취소됨
} else if err != nil {
log.Error(ctx, "Monitoring failed (will retry)", err)
continue // 다음 주기에 재시도
}
}
}
}
```

네트워크 연결 실패 처리

```go
func monitorAccountOnce(ctx context.Context, ...) error {
balance, err := client.BalanceAt(ctx, account.Address, nil)
if err != nil {
return err // 상위 레벨에서 로그 및 재시도 처리
}
```

```
nonce, err := client.NonceAt(ctx, account.Address, nil)
if err != nil {
return err
}
// 메트릭 업데이트는 모든 데이터 수집 후에만 수행
accountBalance.WithLabelValues(chainName, string(account.Role)).Set(bi.ToEtherF6
accountNonce.WithLabelValues(chainName, string(account.Role)).Set(float64(nonce)
return nil
}
```

성능 최적화

1. 배치 처리

여러 메트릭을 한 번에 수집하여 RPC 호출을 최소화합니다:

```
func instrSupplies(ctx context.Context, ...) error {
// 한 번의 쿼리로 모든 공급량 정보 수집
response, err := cprov.QueryClients().Bank.TotalSupply(ctx, &types.QueryTotalSup
if err != nil {
return err
}
// 여러 메트릭을 동시에 업데이트
cChainSupply.Set(bi.ToEtherF64(cosmosSupplyWei))
```

```go
l1Erc20Supply.Set(bi.ToEtherF64(l1TokenSupplyWei))
bridgeBalance.Set(bi.ToEtherF64(l1BridgeBalanceWei))
return nil
}
```

2. 메모리 관리

ResetGaugeVec을 사용하여 동적 라벨의 메모리 누수를 방지합니다:

```go
// 검증자 목록이 변경될 때 기존 메트릭 정리
powerGauge.Reset()
jailedGauge.Reset()
bondedGauge.Reset()

// 새로운 검증자들에 대해 메트릭 설정
for _, val := range vals {
powerGauge.WithLabelValues(val.Address).Set(float64(val.Power))
jailedGauge.WithLabelValues(val.Address).Set(boolToFloat(val.Jailed))
}
```

3. 주기 최적화

컴포넌트별로 적절한 모니터링 주기를 설정합니다:

- **실시간 메트릭**: 10-30초 (블록 헤드, 계정 잔액)

- **통계 메트릭**: 1시간 (스테이킹 보상, 토큰 공급량)
- **상태 메트릭**: 1분 (검증자 상태, 증명 오프셋)

Monitor 서비스의 메트릭 시스템은 Omni 네트워크의 모든 측면을 포괄적으로 모니터링하여 운영자와 개발자가 네트워크 상태를 실시간으로 파악하고 문제 상황에 즉시 대응할 수 있도록 지원합니다.